
■ OpenClaw

AppClaw – Hermes Agent Layer

Complete Swift Source Code
11 source files

File	Description
BearLogoView.swift	OpenClaw bear head logo (SwiftUI)
HermesAgentHarness.swift	Agent roles, harness, typed event stream
HermesContextTracker.swift	Sliding-window topic & intent detector
HermesDreamEngine.swift	Autodream consolidation (5 phases)
HermesIntegration.swift	High-level logging API
HermesKairos.swift	15s budget proactive observer
HermesMemory.swift	On-device memory store + MemoryIndex
HermesProactiveEngine.swift	Always-on suggestion engine
HermesSessionState.swift	Crash-safe state, workflow, token budget
HermesTheme.swift	OpenClaw colour palette & typography
HermesToolRegistry.swift	Metadata-first registry + 18-module security

◆ BearLogoView.swift (253 lines)

```
1 import SwiftUI
2
3 // MARK: - OpenClaw Bear Head Logo
4 //
5 // A cute geometric bear head built entirely from SwiftUI Shapes –
6 // no external assets needed. Scales cleanly from 24 pt (nav bar icon)
7 // up to 512 pt (App Store icon).
8 //
9 // Colour scheme follows the OpenClaw theme: midnight navy background,
10 // amber-claw gradient, electric-blue accent eyes.
11
12 // MARK: - Bear shape primitives
13
14 /// Full bear head silhouette (face + round ears).
15 struct BearHeadShape: Shape {
16     func path(in rect: CGRect) -> Path {
17         let w = rect.width, h = rect.height
18         var p = Path()
19
20         // Main circular face
21         let faceR = w * 0.42
22         let faceC = CGPoint(x: w * 0.50, y: h * 0.54)
23         p.addEllipse(in: CGRect(
24             x: faceC.x - faceR, y: faceC.y - faceR,
25             width: faceR * 2, height: faceR * 2
26         ))
27
28         // Left ear
29         let earR = w * 0.18
30         p.addEllipse(in: CGRect(
31             x: w * 0.12, y: h * 0.06,
32             width: earR * 2, height: earR * 2
33         ))
34
35         // Right ear
36         p.addEllipse(in: CGRect(
37             x: w * 0.70, y: h * 0.06,
38             width: earR * 2, height: earR * 2
39         ))
40
41         return p
42     }
43 }
44
45 /// Inner ear circles.
46 struct BearInnerEarShape: Shape {
47     func path(in rect: CGRect) -> Path {
48         let w = rect.width, h = rect.height
49         var p = Path()
50         let r = w * 0.10
51
52         // Left inner ear
53         p.addEllipse(in: CGRect(x: w * 0.18, y: h * 0.12, width: r * 2, height: r * 2))
54         // Right inner ear
55         p.addEllipse(in: CGRect(x: w * 0.72, y: h * 0.12, width: r * 2, height: r * 2))
56         return p
57     }
58 }
59
60 /// Snout area – slightly lighter oval below the eyes.
61 struct BearSnoutShape: Shape {
62     func path(in rect: CGRect) -> Path {
63         let w = rect.width, h = rect.height
64         var p = Path()
65         let sw = w * 0.32, sh = h * 0.20
66         p.addEllipse(in: CGRect(
67             x: (w - sw) / 2, y: h * 0.60,
68             width: sw, height: sh
69         ))
70         return p
71     }
72 }
73
74 // MARK: - BearLogoView
75
```

```

76 /// Compositing bear head with OpenGL theming.
77 /// - Parameters:
78 /// - size: Bounding square in points (default 64).
79 /// - showBackground: If true, draws a rounded-rect card background.
80 struct BearLogoView: View {
81     var size: CGFloat = 64
82     var showBackground: Bool = true
83
84     var body: some View {
85         ZStack {
86             // Optional card background
87             if showBackground {
88                 RoundedRectangle(cornerRadius: size * 0.22)
89                 .fill(
90                     LinearGradient(
91                         colors: [Color(hex: "#1A2233"), Color.OC.background],
92                         startPoint: .topLeading,
93                         endPoint: .bottomTrailing
94                     )
95                 )
96                 .frame(width: size, height: size)
97                 .shadow(color: Color.OC.accent.opacity(0.3), radius: size * 0.12, y: size * 0.04)
98             }
99
100             // Bear head silhouette – amber-claw gradient
101             BearHeadShape()
102             .fill(
103                 LinearGradient(
104                     colors: [Color(hex: "#C8862A"), Color(hex: "#FF9F0A"), Color(hex: "#E8A030")],
105                     startPoint: .topLeading,
106                     endPoint: .bottomTrailing
107                 )
108             )
109             .frame(width: size * 0.82, height: size * 0.82)
110
111             // Inner ears – darker amber
112             BearInnerEarShape()
113             .fill(Color(hex: "#A06820"))
114             .frame(width: size * 0.82, height: size * 0.82)
115
116             // Snout – cream
117             BearSnoutShape()
118             .fill(Color(hex: "#F0D090"))
119             .frame(width: size * 0.82, height: size * 0.82)
120
121             // Eyes – electric blue with specular dot
122             bearEyes
123
124             // Nose – small dark oval on snout
125             Ellipse()
126             .fill(Color(hex: "#3A2010"))
127             .frame(width: size * 0.10, height: size * 0.065)
128             .offset(y: size * 0.185)
129
130             // Subtle "claw mark" overlay – three thin diagonal lines at bottom-right
131             if size >= 40 {
132                 clawMarkOverlay
133             }
134         }
135         .frame(width: size, height: size)
136     }
137
138     // MARK: - Eyes
139
140     @ViewBuilder
141     private var bearEyes: some View {
142         let eyeR = size * 0.068
143         let eyeOffX = size * 0.135
144         let eyeOffY = size * 0.04
145
146         ZStack {
147             // Left eye white
148             Circle().fill(Color.white)
149             .frame(width: eyeR * 2.2, height: eyeR * 2.2)
150             .offset(x: -eyeOffX, y: -eyeOffY)
151             // Left iris – electric blue
152             Circle()

```

```

153 .fill(
154   RadialGradient(
155     colors: [Color.OC.primary, Color(hex: "#0055CC")],
156     center: .center, startRadius: 0, endRadius: eyeR
157   )
158 )
159 .frame(width: eyeR * 2, height: eyeR * 2)
160 .offset(x: -eyeOffX, y: -eyeOffY)
161 // Left specular
162 Circle().fill(Color.white.opacity(0.7))
163 .frame(width: eyeR * 0.55, height: eyeR * 0.55)
164 .offset(x: -eyeOffX + eyeR * 0.35, y: -eyeOffY - eyeR * 0.35)
165
166 // Right eye white
167 Circle().fill(Color.white)
168 .frame(width: eyeR * 2.2, height: eyeR * 2.2)
169 .offset(x: eyeOffX, y: -eyeOffY)
170 // Right iris
171 Circle()
172 .fill(
173   RadialGradient(
174     colors: [Color.OC.primary, Color(hex: "#0055CC")],
175     center: .center, startRadius: 0, endRadius: eyeR
176   )
177 )
178 .frame(width: eyeR * 2, height: eyeR * 2)
179 .offset(x: eyeOffX, y: -eyeOffY)
180 // Right specular
181 Circle().fill(Color.white.opacity(0.7))
182 .frame(width: eyeR * 0.55, height: eyeR * 0.55)
183 .offset(x: eyeOffX + eyeR * 0.35, y: -eyeOffY - eyeR * 0.35)
184 }
185 }
186
187 // MARK: - Claw mark
188
189 @ViewBuilder
190 private var clawMarkOverlay: some View {
191   let s = size
192   // Bug fix #1: use the Canvas closure's own `canvasSize` instead of the
193   // captured `s` — avoids misalignment when layout resolves a different size.
194   Canvas { ctx, canvasSize in
195     let w = canvasSize.width, h = canvasSize.height
196     let offX = w * 0.26
197     let offY = h * 0.22
198     let len = h * 0.18
199     let gap = w * 0.055
200     for i in 0..<3 {
201       let dx = CGFloat(i) * gap
202       var line = Path()
203       line.move(to: CGPoint(x: w / 2 + offX + dx, y: h / 2 + offY))
204       line.addLine(to: CGPoint(x: w / 2 + offX + dx + len * 0.5, y: h / 2 + offY + len))
205       ctx.stroke(line,
206         with: .color(Color.OC.accent.opacity(0.45)),
207         style: StrokeStyle(lineWidth: w * 0.022, lineCap: .round))
208     }
209   }
210   .frame(width: size, height: size)
211   .blendMode(.overlay)
212 }
213 }
214
215 // MARK: - App icon variant (no background card, for Asset Catalog)
216
217 struct BearIconView: View {
218   var size: CGFloat = 1024
219   var body: some View {
220     ZStack {
221       // Solid background for App Store icon
222       RoundedRectangle(cornerRadius: size * 0.2237) // Apple icon corner formula
223       .fill(
224         LinearGradient(
225           colors: [Color(hex: "#131A25"), Color(hex: "#0D1117")],
226           startPoint: .top, endPoint: .bottom
227         )
228       )
229       BearLogoView(size: size * 0.75, showBackground: false)

```

```
230 }
231 .frame(width: size, height: size)
232 }
233 }
234
235 // MARK: - Preview
236
237 #if DEBUG
238 #Preview("Logo sizes") {
239   HStack(spacing: 20) {
240     ForEach([24, 44, 64, 96], id: \.self) { sz in
241       BearLogoView(size: CGFloat(sz))
242     }
243   }
244   .padding(24)
245   .background(Color.OC.background)
246 }
247
248 #Preview("App icon") {
249   BearIconView(size: 256)
250   .padding(20)
251   .background(Color.OC.background)
252 }
253 #endif
```

◆ HermesAgentHarness.swift (323 lines)

```
1 import Foundation
2
3 // MARK: - Agent Role System
4 //
5 // Each role has sharply-defined behavioural constraints and a restricted
6 // tool pool. Specialisation prevents role-bleed where an exploration
7 // agent accidentally overwrites memory, or an execution agent skips
8 // planning.
9
10 enum AgentRole: String, Codable, CaseIterable {
11 case explore // read-only investigation; never writes
12 case plan // synthesises findings into a step list; no side effects
13 case execute // carries out the plan; only role with write access
14 case verify // confirms execute's output; read-only, high scrutiny
15
16 var allowedTier: PermissionTier {
17 switch self {
18 case .explore: return .readonly
19 case .plan: return .readonly
20 case .execute: return .privileged
21 case .verify: return .readonly
22 }
23 }
24
25 var description: String {
26 switch self {
27 case .explore: return "Read-only investigation of memory and context."
28 case .plan: return "Synthesise findings into an ordered step list."
29 case .execute: return "Carry out the plan with write access."
30 case .verify: return "Confirm execution results and flag discrepancies."
31 }
32 }
33 }
34
35 // MARK: - Typed Event Stream
36 //
37 // Every harness operation emits a typed AgentEvent. These events are the
38 // single source of truth for logging, debugging, and dual-level verification.
39 //
40 // Bug fix #2: `Error` does not conform to `Sendable`, so `AgentEvent` must
41 // be `@unchecked Sendable` to cross actor boundaries under Swift 6 strict
42 // concurrency. We own all construction sites, so this is safe.
43
44 enum AgentEvent: @unchecked Sendable {
45 case sessionStarted(conversationId: String, role: AgentRole)
46 case toolSelected(toolId: String, role: AgentRole)
47 case toolValidated(toolId: String, result: HermesToolRegistry.PermissionResult)
48 case toolExecuted(toolId: String, success: Bool, durationMs: Int)
49 case budgetChecked(remaining: Int, estimatedCost: Int, allowed: Bool)
50 case transcriptCompacted(beforeTokens: Int, afterTokens: Int)
51 case workflowStepCompleted(stepName: String, index: Int, total: Int)
52 case verificationPassed(checkName: String)
53 case verificationFailed(checkName: String, reason: String)
54 case harnessError(Error)
55 }
56
57 // MARK: - Transcript Message
58
59 struct TranscriptMessage: Codable {
60 enum Role: String, Codable { case user, assistant, system }
61 let role: Role
62 let content: String
63 let timestamp: Date
64 var tokenEstimate: Int // rough: content.count / 4
65 var isPinned: Bool = false // pinned messages survive compaction
66
67 init(role: Role, content: String, timestamp: Date = Date()) {
68 self.role = role
69 self.content = content
70 self.timestamp = timestamp
71 self.tokenEstimate = max(1, content.count / 4)
72 }
73 }
74
75 // MARK: - HermesAgentHarness
```

```

76
77 /// Ties all subsystems together. Entry point for all agentic operations.
78 /// Use `run(role:task:)` to dispatch a scoped, permission-bounded agent.
79 actor HermesAgentHarness {
80 static let shared = HermesAgentHarness()
81
82 private let memory = HermesMemory.shared
83 private let registry = HermesToolRegistry.shared
84 private let session = HermesSessionState.shared
85 private let kairos = HermesKairos.shared
86 private let context = HermesContextTracker.shared
87
88 private var eventLog: [AgentEvent] = []
89 private var transcript: [TranscriptMessage] = []
90
91 // Token ceiling before auto-compaction triggers
92 private let compactionThreshold = 6_000
93
94 private init() {}
95
96 // MARK: - Agent dispatch
97
98 /// Run a scoped agent. Returns a structured result and emits events throughout.
99 func run(role: AgentRole, task: String) async throws -> AgentResult {
100 let conversationId = await session.conversation.id
101
102 emit(.sessionStarted(conversationId: conversationId, role: role))
103
104 // Pre-turn token budget check
105 let budget = await session.tokenBudget
106 let allowed = budget.canStartTurn(estimatedCost: 500)
107 emit(.budgetChecked(remaining: budget.remaining, estimatedCost: 500, allowed: allowed))
108 guard allowed else { throw SessionError.tokenBudgetExhausted(used: budget.used, limit: budget.sessionLimit) }
109
110 // Build context-appropriate tool pool
111 let contextTags = Set((await context.currentTopic() ?? "").components(separatedBy: ", "))
112 let pool = await registry.assemblePool(for: role, contextTags: contextTags)
113
114 // Record the task in the transcript
115 append(message: TranscriptMessage(role: .user, content: task))
116
117 // Auto-compact if approaching threshold
118 if transcriptTokenCount() > compactionThreshold {
119 await compactTranscript()
120 }
121
122 // Build permission context for this agent
123 let permCtx = PermissionContext(
124 sessionTier: role.allowedTier,
125 isInForeground: true,
126 tokenBudgetRemaining: budget.remaining,
127 kairosActive: false,
128 metadata: ["agentRole": role.rawValue]
129 )
130
131 // Execute tools in pool order (explore/plan are read-only; execute writes)
132 var toolResults: [String: Any] = [:]
133 for tool in pool {
134 emit(.toolSelected(toolId: tool.id, role: role))
135
136 let validation = await registry.validate(tool: tool.id, context: permCtx)
137 emit(.toolValidated(toolId: tool.id, result: validation))
138 guard validation.isAllowed else { continue }
139
140 let start = Date()
141 let result = await executeTool(tool, context: permCtx)
142 let ms = Int(Date().timeIntervalSince(start) * 1000)
143 emit(.toolExecuted(toolId: tool.id, success: result.success, durationMs: ms))
144
145 if result.success { toolResults[tool.id] = result.value }
146 }
147
148 // Build a response and append to transcript
149 let response = synthesiseResponse(role: role, task: task, toolResults: toolResults)
150 append(message: TranscriptMessage(role: .assistant, content: response))
151
152 // Dual-level verification

```

```

153 let agentVerification = verifyAgentOutput(response: response, role: role)
154 let harnessVerification = verifyHarness(toolResults: toolResults, role: role)
155
156 for check in agentVerification + harnessVerification {
157 emit(check.passed
158 ? .verificationPassed(checkName: check.name)
159 : .verificationFailed(checkName: check.name, reason: check.reason ?? ""))
160 }
161
162 // Record token usage (rough estimate)
163 let promptEst = transcriptTokenCount()
164 let replyEst = max(1, response.count / 4)
165 try await session.recordTokenUsage(prompt: promptEst, completion: replyEst)
166
167 return AgentResult(
168 role: role,
169 response: response,
170 toolsUsed: Array(toolResults.keys),
171 verificationPassed: (agentVerification + harnessVerification).allSatisfy(\.passed),
172 events: eventLog
173 )
174 }
175
176 // MARK: - Tool execution (stub – wire up real tool handlers here)
177
178 private func executeTool(_ tool: ToolDefinition,
179 context: PermissionContext) async -> ToolResult {
180 switch tool.id {
181 case "hermes.memory.search":
182 return ToolResult(success: true, value: "memory_results_placeholder")
183 case "hermes.context.topic":
184 let topic = await self.context.currentTopic() ?? "unknown"
185 return ToolResult(success: true, value: topic)
186 case "hermes.dream.trigger":
187 await HermesDreamEngine.shared.runDreamCycle()
188 return ToolResult(success: true, value: "dream_complete")
189 case "hermes.suggestions.refresh":
190 await HermesProactiveEngine.shared.refresh()
191 return ToolResult(success: true, value: "suggestions_refreshed")
192 default:
193 return ToolResult(success: false, value: nil)
194 }
195 }
196
197 // MARK: - Transcript compaction
198 //
199 // Keeps pinned messages and the most recent tail intact.
200 // Summarises older messages into a single system-role placeholder.
201 // This prevents context entropy while staying within token limits.
202
203 private func compactTranscript() async {
204 let before = transcriptTokenCount()
205 let pinned = transcript.filter(\.isPinned)
206 let unpinned = transcript.filter { !$0.isPinned }
207
208 // Keep last 10 unpinned messages; summarise the rest
209 let keep = Array(unpinned.suffix(10))
210 let summarised = unpinned.dropLast(10)
211
212 if summarised.isEmpty { return }
213
214 let summary = "[Compacted \(summarised.count) earlier messages. " +
215 "Topics covered: \(topicSummary(of: summarised))]"
216 let placeholder = TranscriptMessage(role: .system, content: summary)
217
218 transcript = pinned + [placeholder] + keep
219 let after = transcriptTokenCount()
220 emit(.transcriptCompacted(beforeTokens: before, afterTokens: after))
221 }
222
223 private func topicSummary(of messages: [TranscriptMessage]) -> String {
224 let words = messages.map(\.content).joined(separator: " ")
225 .lowercased()
226 .components(separatedBy: .alphanumerics.inverted)
227 .filter { $0.count > 4 }
228 let freq = Dictionary(words.map { ($0, 1) }, uniquingKeysWith: +)
229 return freq.sorted { $0.value > $1.value }.prefix(5).map(\.key).joined(separator: ", ")

```



```

230 }
231
232 private func transcriptTokenCount() -> Int {
233 transcript.reduce(0) { $0 + $1.tokenEstimate }
234 }
235
236 private func append(message: TranscriptMessage) {
237 transcript.append(message)
238 }
239
240 // MARK: - Response synthesis
241
242 private func synthesiseResponse(role: AgentRole,
243 task: String,
244 toolResults: [String: Any]) -> String {
245 // Bug fix #7: assert in debug so this stub is never silently shipped.
246 // TODO: replace with real LLM API call using HermesSessionState.tokenBudget.
247 #if DEBUG
248 // assertionFailure("synthesiseResponse is a stub – wire up your LLM API here.")
249 #endif
250 let toolSummary = toolResults.isEmpty
251 ? "No tools returned results."
252 : toolResults.keys.joined(separator: ", ")
253 return "[\(role.rawValue.uppercased())] Task: \(task). Tools used: \(toolSummary)."
254 }
255
256 // MARK: - Dual-level verification
257
258 private struct VerificationCheck {
259 let name: String
260 let passed: Bool
261 let reason: String?
262 }
263
264 /// Level 1 – verify the agent's own output (content checks).
265 private func verifyAgentOutput(response: String, role: AgentRole) -> [VerificationCheck] {
266 [
267 VerificationCheck(
268 name: "response_non_empty",
269 passed: !response.isEmpty,
270 reason: response.isEmpty ? "Agent produced an empty response." : nil
271 ),
272 VerificationCheck(
273 name: "no_hardcoded_secrets",
274 passed: !response.lowercased().contains("api_key") &&
275 !response.lowercased().contains("password"),
276 reason: "Response may contain sensitive literals."
277 ),
278 ]
279 }
280
281 /// Level 2 – verify the harness (structural / permission checks).
282 private func verifyHarness(toolResults: [String: Any], role: AgentRole) -> [VerificationCheck] {
283 let writeToolsUsed = toolResults.keys.filter {
284 $0.contains("write") || $0.contains("observe") || $0.contains("dream")
285 }
286 return [
287 VerificationCheck(
288 name: "no_writes_in_readonly_role",
289 passed: role != .explore || writeToolsUsed.isEmpty,
290 reason: writeToolsUsed.isEmpty ? nil
291 : "Explore role invoked write tool(s): \(writeToolsUsed.joined(separator: ", "))"
292 ),
293 ]
294 }
295
296 // MARK: - Event log
297
298 private func emit(_ event: AgentEvent) {
299 eventLog.append(event)
300 #if DEBUG
301 print("[Harness] \(event)")
302 #endif
303 }
304
305 func recentEvents(limit: Int = 50) -> [AgentEvent] {
306 Array(eventLog.suffix(limit))

```


◆ HermesContextTracker.swift (139 lines)

```
1 import Foundation
2
3 // MARK: - Role
4
5 enum MessageRole {
6 case user
7 case assistant
8 }
9
10 // MARK: - TrackedMessage
11
12 private struct TrackedMessage {
13 let role: MessageRole
14 let text: String
15 let timestamp: Date
16 }
17
18 // MARK: - HermesContextTracker
19 //
20 // Maintains a sliding window of recent messages and extracts the current
21 // topic + intent without any network calls.
22 //
23 // Used by HermesIntegration.currentTopic() and HermesProactiveEngine
24 // for richer, more accurate suggestions.
25
26 actor HermesContextTracker {
27 static let shared = HermesContextTracker()
28
29 /// How many messages to keep in the active window.
30 private let windowSize = 20
31
32 private var window: [TrackedMessage] = []
33
34 // Cache so we don't re-derive on every call
35 private var cachedTopic: String?
36 private var topicDirty = true
37
38 private init() {}
39
40 // MARK: - Ingestion
41
42 /// Add a new message to the sliding window.
43 func ingest(text: String, role: MessageRole) {
44 let msg = TrackedMessage(role: role, text: text, timestamp: Date())
45 window.append(msg)
46 if window.count > windowSize { window.removeFirst() }
47 topicDirty = true
48 }
49
50 // MARK: - Derived context
51
52 /// Best-guess "current topic" from the recent window.
53 /// Returns nil if the window is empty.
54 func currentTopic() -> String? {
55 guard !window.isEmpty else { return nil }
56 if !topicDirty, let cached = cachedTopic { return cached }
57 let topic = deriveTopic()
58 cachedTopic = topic
59 topicDirty = false
60 return topic
61 }
62
63 /// Rough intent category of the most recent user message.
64 func currentIntent() -> Intent? {
65 guard let lastUser = window.last(where: { $0.role == .user }) else { return nil }
66 return classifyIntent(lastUser.text)
67 }
68
69 /// Plain summary of the window – useful for injecting context into a new conversation.
70 func sessionSummary() -> String {
71 let userLines = window
72 .filter { $0.role == .user }
73 .suffix(5)
74 .map { "• \"\($0.text.prefix(120))\" }
75 .joined(separator: "\n")
76 }
```

```

76 guard !userLines.isEmpty else { return "No activity yet." }
77 return "Recent messages:\n\u{000A}(userLines)"
78 }
79
80 // MARK: - Topic derivation
81
82 private static let stopwords: Set<String> = [
83 "the","and","for","are","but","not","you","all","can","had","her","was","one",
84 "our","out","day","get","has","him","his","how","man","new","now","old","see",
85 "two","way","who","did","its","let","put","say","she","too","use","that","with",
86 "have","this","will","your","from","they","know","want","been","good","much",
87 "some","time","very","when","come","here","just","like","long","make","many",
88 "more","only","over","such","take","than","them","well","were","what","also",
89 "into","most","then","there","these","think","those","about","after","being",
90 "could","going","their","where","which","would","should","because"
91 ]
92
93 private func deriveTopic() -> String? {
94 // Weight user messages 2x over assistant messages
95 let corpus = window.map { msg -> String in
96 let weight = msg.role == .user ? 2 : 1
97 return Array(repeating: msg.text, count: weight).joined(separator: " ")
98 }.joined(separator: " ")
99
100 let words = corpus
101 .lowercased()
102 .components(separatedBy: .alphanumerics.inverted)
103 .filter { $0.count > 3 && !Self.stopwords.contains($0) }
104
105 let freq = Dictionary(words.map { ($0, 1) }, uniquingKeysWith: +)
106 let top = freq.sorted { $0.value > $1.value }.prefix(3).map(\.key)
107 return top.isEmpty ? nil : top.joined(separator: ", ")
108 }
109
110 // MARK: - Intent classification
111
112 enum Intent: String {
113 case question // asking for information
114 case debugRequest // fixing a bug or error
115 case buildRequest // creating something new
116 case explainRequest // asking for an explanation
117 case editRequest // modifying existing content
118 case other
119 }
120
121 private func classifyIntent(_ text: String) -> Intent {
122 let t = text.lowercased()
123
124 let debugSignals = ["error","bug","fix","crash","fail","broken","not working","exception","issue"]
125 let buildSignals = ["create","build","make","add","implement","generate","write","new"]
126 let explainSignals = ["explain","what is","what are","how does","why does","tell me","describe"]
127 let editSignals = ["change","update","modify","rename","refactor","move","delete","remove"]
128 let questionSignals = ["?","how","what","when","where","who","which","why","can you"]
129
130 func matches(_ signals: [String]) -> Bool { signals.contains { t.contains($0) } }
131
132 if matches(debugSignals) { return .debugRequest }
133 if matches(buildSignals) { return .buildRequest }
134 if matches(explainSignals) { return .explainRequest }
135 if matches(editSignals) { return .editRequest }
136 if matches(questionSignals){ return .question }
137 return .other
138 }
139 }

```

◆ HermesDreamEngine.swift (318 lines)

```
1 import Foundation
2 import BackgroundTasks
3
4 // MARK: - HermesDreamEngine
5 //
6 // "Dream mode" – runs while the app is backgrounded, targeting ~2 am local time.
7 //
8 // Improvements over v1:
9 // - Batch memory updates: all promotions in one persist call (was N separate calls)
10 // - Smart 2 am scheduling: aims for the next 2 am local time, not "5 hours from now"
11 // - Proper English stopword list for topic insight generation
12 // - Removed unused `dreamLog: [DreamEntry]`
13 // - `handleDream` no longer captures `self` strongly in nested closure
14 // - `phase4_selfImprove` scoped to recent 24 h, not all-time
15
16 final class HermesDreamEngine {
17     static let shared = HermesDreamEngine()
18
19     private let taskIdentifier = "com.openclaw.hermes.dream"
20     private let memory = HermesMemory.shared
21
22     private init() {}
23
24     // MARK: - Registration
25
26     func registerBackgroundTask() {
27         BGTaskScheduler.shared.register(forTaskWithIdentifier: taskIdentifier, using: nil) { [weak self] task in
28             guard let self, let processingTask = task as? BGProcessingTask else {
29                 task.setTaskCompleted(success: false)
30                 return
31             }
32             self.handleDream(task: processingTask)
33         }
34     }
35
36     /// Call when the app moves to background. Schedules the next dream at ~2 am local.
37     func scheduleNextDream() {
38         let request = BGProcessingTaskRequest(identifier: taskIdentifier)
39         request.requiresNetworkConnectivity = false
40         request.requiresExternalPower = false
41         request.earliestBeginDate = nextDreamDate()
42         try? BGTaskScheduler.shared.submit(request)
43     }
44
45     /// Next occurrence of 2 am local time. If it's already past 2 am today, uses tomorrow.
46     private func nextDreamDate() -> Date {
47         let cal = Calendar.current
48         var components = cal.dateComponents([.year, .month, .day], from: Date())
49         components.hour = 2
50         components.minute = 0
51         components.second = 0
52         let todayAt2 = cal.date(from: components) ?? Date()
53         // If we've passed today's 2 am, aim for tomorrow
54         return todayAt2 > Date() ? todayAt2 : todayAt2.addingTimeInterval(86400)
55     }
56
57     // MARK: - Dream execution
58
59     private func handleDream(task: BGProcessingTask) {
60         let dreamTask = Task { [weak self] in
61             guard let self else { return }
62             await self.runDreamCycle()
63             task.setTaskCompleted(success: true)
64             self.scheduleNextDream()
65         }
66         task.expirationHandler = { dreamTask.cancel() }
67     }
68
69     /// Full dream cycle. Also callable manually (e.g., from a Debug/Settings screen).
70     @discardableResult
71     func runDreamCycle() async -> DreamReport {
72         var report = DreamReport(startedAt: Date())
73
74         await phase1_consolidate(&report)
75         await phase2_promotePatterns(&report)
```

```

76 await phase3_generateInsights(&report)
77 await phase4_selfImprove(&report)
78 await phase5_resolveContradictions(&report)
79
80 report.finishedAt = Date()
81 await saveDreamReport(report)
82 return report
83 }
84
85 // MARK: - Phase 1: Consolidate
86
87 private func phase1_consolidate(_ report: inout DreamReport) async {
88 let before = await memory.allEntries().count
89 do {
90 try await memory.consolidate(importanceThreshold: 2, keepWindow: 7 * 86400)
91 let after = await memory.allEntries().count
92 report.pruned = max(0, before - after)
93 report.phases.append("Pruned \(report.pruned) low-importance entries.")
94 } catch {
95 report.phases.append("Consolidation error: \(error)")
96 }
97 }
98
99 // MARK: - Phase 2: Promote patterns (single batch persist)
100
101 private func phase2_promotePatterns(_ report: inout DreamReport) async {
102 let all = await memory.allEntries()
103
104 let freq = Dictionary(grouping: all, by: \.category).mapValues(\.count)
105
106 // Collect all mutations first, then write once
107 var updated: [MemoryEntry] = []
108 var promotedIDs: [UUID] = []
109
110 for entry in all {
111 let count = freq[entry.category, default: 0]
112 if count > 5 && entry.importance < 4 {
113 var e = entry
114 e = MemoryEntry(
115 id: e.id, timestamp: e.timestamp, category: e.category,
116 content: e.content.value,
117 metadata: e.metadata.mapValues(\.value),
118 importance: min(e.importance + 1, 5),
119 tier: e.tier
120 )
121 updated.append(e)
122 }
123 // Promote high-importance entries to long-term tier
124 if entry.importance >= 4 && entry.tier == .shortTerm {
125 promotedIDs.append(entry.id)
126 }
127 }
128
129 do {
130 if !updated.isEmpty { try await memory.updateBatch(updated) }
131 if !promotedIDs.isEmpty { try await memory.promoteToLongTerm(promotedIDs) }
132 } catch {
133 report.phases.append("Promotion error: \(error)")
134 }
135
136 report.promoted = updated.count
137 report.phases.append("Promoted \(updated.count) entries; moved \(promotedIDs.count) to long-term.")
138 }
139
140 // MARK: - Phase 3: Generate insights
141
142 private static let stopwords: Set<String> = [
143 "the", "and", "for", "are", "but", "not", "you", "all", "can", "had", "her", "was", "one",
144 "our", "out", "day", "get", "has", "him", "his", "how", "man", "new", "now", "old", "see",
145 "two", "way", "who", "boy", "did", "its", "let", "put", "say", "she", "too", "use", "that",
146 "with", "have", "this", "will", "your", "from", "they", "know", "want", "been", "good",
147 "much", "some", "time", "very", "when", "come", "here", "just", "like", "long", "make",
148 "many", "more", "only", "over", "such", "take", "than", "them", "well", "were", "what",
149 "also", "into", "most", "other", "said", "then", "there", "these", "think", "those",
150 "about", "after", "being", "could", "every", "going", "great", "their", "where", "which",
151 "would", "should", "would", "because", "through"
152 ]

```

```

153
154 private func phase3_generateInsights(_ report: inout DreamReport) async {
155     let recentMessages = await memory.entries(for: "user_message")
156     .prefix(100)
157     .compactMap { ($0.content.value as? [String: Any])?["text"] as? String }
158
159     guard !recentMessages.isEmpty else {
160         report.phases.append("No recent messages to analyse.")
161         return
162     }
163
164     let words = recentMessages.joined(separator: " ")
165     .lowercased()
166     .components(separatedBy: .alphanumerics.inverted)
167     .filter { $0.count > 3 && !Self.stopwords.contains($0) }
168
169     let freq = Dictionary(words.map { ($0, 1) }, uniquingKeysWith: +)
170     let topWords = freq.sorted { $0.value > $1.value }.prefix(8).map{\.key}
171
172     guard !topWords.isEmpty else { return }
173
174     let insight = "Recurring topics in recent conversations: \(topWords.joined(separator: ", "))."
175     report.insights.append(insight)
176     report.phases.append("Topic insight: \(insight)")
177
178     try? await memory.observe(
179         category: "dream_insight",
180         content: ["insight": insight, "basedOn": recentMessages.count, "topWords": topWords],
181         metadata: ["importance": 4]
182     )
183 }
184
185 // MARK: - Phase 4: Self-improve
186
187 private func phase4_selfImprove(_ report: inout DreamReport) async {
188     // Scope to last 24 h only – all-time error counts are noisy
189     let since = Date().addingTimeInterval(-86400)
190     let errors = await memory.entries(for: "tool_exec")
191     .filter { $0.timestamp >= since }
192     .filter {
193         guard let d = $0.content.value as? [String: Any],
194             let ok = d["success"] as? Bool else { return false }
195         return !ok
196     }
197
198     guard errors.count >= 3 else {
199         report.phases.append("Self-improvement: no recurring failures in last 24 h.")
200         return
201     }
202
203     let toolNames = errors.compactMap { ($0.content.value as? [String: Any])?["tool"] as? String }
204     let counts = Dictionary(toolNames.map { ($0, 1) }, uniquingKeysWith: +)
205     let worst = counts.sorted { $0.value > $1.value }.prefix(2)
206     .map { "\($0.key) (\($0.value)x)" }
207     let note = "Heads-up: repeated failures in \(worst.joined(separator: ", ")) over the last 24 h. Consider revi...
208
209     report.improvements.append(note)
210     report.phases.append("Self-improvement note written.")
211
212     try? await memory.observe(
213         category: "self_improvement",
214         content: ["note": note, "errorCount": errors.count],
215         metadata: ["importance": 5]
216     )
217 }
218
219 // MARK: - Phase 5: Resolve contradictions
220 //
221 // Looks for entries in the same category where the content directly
222 // contradicts a later entry (e.g. two self_improvement notes about the
223 // same tool with conflicting advice, or two dream_insights with
224 // opposite conclusions).
225 //
226 // Strategy: when two entries in the same category share the same top
227 // keyword AND were written > 1 h apart, keep only the newer one and
228 // mark the older as superseded (importance → 1 so it gets pruned next cycle).
229

```

```

230 private func phase5_resolveContradictions(_ report: inout DreamReport) async {
231     let candidateCategories = ["dream_insight", "self_improvement", "kairos_insight"]
232     var resolved = 0
233
234     for category in candidateCategories {
235         let entries = await memory.entries(for: category)
236         guard entries.count >= 2 else { continue }
237
238         // Group by dominant keyword (first non-stopword word in content text)
239         var groups: [String: [MemoryEntry]] = [:]
240         for entry in entries {
241             let text = (entry.content.value as? [String: Any])?
242                 .values.compactMap { $0 as? String }.joined(separator: " ")
243             ?? "\(entry.content.value)"
244             if let key = dominantKeyword(text) {
245                 groups[key, default: []].append(entry)
246             }
247         }
248
249         for (_, group) in groups where group.count >= 2 {
250             // Sort newest first; demote all but the newest
251             let sorted = group.sorted { $0.timestamp > $1.timestamp }
252             let toSupersede = sorted.dropFirst()
253             .filter { $0.importance > 1 }
254
255             let superseded: [MemoryEntry] = toSupersede.map { entry in
256                 MemoryEntry(
257                     id: entry.id, timestamp: entry.timestamp,
258                     category: entry.category,
259                     content: entry.content.value,
260                     metadata: entry.metadata.mapValues { $0.value },
261                     importance: 1, // will be pruned next consolidation
262                     tier: entry.tier
263                 )
264             }
265
266             if !superseded.isEmpty {
267                 try? await memory.updateBatch(superseded)
268                 resolved += superseded.count
269             }
270         }
271     }
272
273     report.resolved = resolved
274     report.phases.append(
275         resolved > 0
276         ? "Contradiction resolution: superseded \(resolved) outdated entries."
277         : "Contradiction resolution: no contradictions found."
278     )
279 }
280
281 private func dominantKeyword(_ text: String) -> String? {
282     text.lowercased()
283     .components(separatedBy: .alphanumerics.inverted)
284     .filter { $0.count > 4 && !Self.stopwords.contains($0) }
285     .first
286 }
287
288 // MARK: - Persistence
289
290 private func saveDreamReport(_ report: DreamReport) async {
291     let duration = report.finishedAt.map { $0.timeIntervalSince(report.startedAt) } ?? 0
292     try? await memory.observe(
293         category: "dream_report",
294         content: [
295             "duration_s": Int(duration),
296             "pruned": report.pruned,
297             "promoted": report.promoted,
298             "resolved": report.resolved,
299             "insights": report.insights,
300             "improvements": report.improvements
301         ],
302         metadata: ["importance": 4]
303     )
304 }
305 }
306

```

```
307 // MARK: - Models
308
309 struct DreamReport {
310     let startedAt: Date
311     var finishedAt: Date?
312     var pruned: Int = 0
313     var promoted: Int = 0
314     var resolved: Int = 0
315     var phases: [String] = []
316     var insights: [String] = []
317     var improvements: [String] = []
318 }
```

◆ HermesIntegration.swift (141 lines)

```
1 import Foundation
2
3 // MARK: - HermesSession
4
5 /// Lightweight descriptor of one app session (open → background/close).
6 struct HermesSession {
7     let id: String
8     let startedAt: Date
9     var messageCount: Int = 0
10    var toolCallCount: Int = 0
11 }
12
13 // MARK: - HermesIntegration
14 //
15 // High-level API for the rest of AppClaw.
16 // Fully local – no HTTP, no server, no network permission needed.
17 //
18 // Additions over v1:
19 // - Session lifecycle: logSessionStart / logSessionEnd
20 // - messageCount / toolCallCount tracked per session
21 // - Exposes currentSession for context-aware UI
22
23 actor HermesIntegration {
24     static let shared = HermesIntegration()
25
26     private let memory = HermesMemory.shared
27     private let proactive = HermesProactiveEngine.shared
28     private let contextTracker = HermesContextTracker.shared
29
30     private(set) var lastError: Error?
31     private(set) var currentSession: HermesSession?
32
33     private init() {}
34
35     // MARK: - Session lifecycle
36
37     /// Call when a new conversation starts (app launch, new chat, scene activation).
38     func logSessionStart(conversationId: String) async {
39         let session = HermesSession(id: conversationId, startedAt: Date())
40         currentSession = session
41         await run {
42             try await self.memory.observe(
43                 category: "session_start",
44                 content: ["conversationId": conversationId],
45                 metadata: ["importance": 2]
46             )
47         }
48     }
49
50     /// Call when the app backgrounds or a conversation ends.
51     func logSessionEnd() async {
52         guard let session = currentSession else { return }
53         currentSession = nil
54         await run {
55             try await self.memory.observe(
56                 category: "session_end",
57                 content: [
58                     "conversationId": session.id,
59                     "durationSeconds": Int(Date().timeIntervalSince(session.startedAt)),
60                     "messageCount": session.messageCount,
61                     "toolCallCount": session.toolCallCount
62                 ],
63                 metadata: ["importance": 3]
64             )
65         }
66     }
67
68     // MARK: - Logging
69
70     func logUserMessage(_ text: String, in conversationId: String) async {
71         currentSession?.messageCount += 1
72         await contextTracker.ingest(text: text, role: .user)
73         await run {
74             try await self.memory.observe(
75                 category: "user_message",
```

```

76 content: ["text": text, "conversationId": conversationId],
77 metadata: ["importance": 3]
78 )
79 }
80 }
81
82 func logAssistantResponse(_ text: String, toolUses: [String] = []) async {
83     await contextTracker.ingest(text: text, role: .assistant)
84     await run {
85         try await self.memory.observe(
86             category: "assistant_response",
87             content: ["text": text, "tools": toolUses]
88         )
89     }
90 }
91
92 func logToolExecution(_ toolName: String, success: Bool, duration: TimeInterval) async {
93     currentSession?.toolCallCount += 1
94     await run {
95         try await self.memory.observe(
96             category: "tool_exec",
97             content: [
98                 "tool": toolName,
99                 "success": success,
100                 "duration_ms": Int(duration * 1000)
101             ],
102             // Failures matter more: keep them longer, surface in DreamEngine
103             metadata: ["importance": success ? 1 : 3]
104         )
105     }
106 }
107
108 // MARK: - Queries
109
110 func pollSuggestions() async -> [HermesSuggestion] {
111     await proactive.currentSuggestions()
112 }
113
114 func currentTopic() async -> String? {
115     await contextTracker.currentTopic()
116 }
117
118 func searchMemory(query: String, limit: Int = 20) async -> [MemoryEntry] {
119     await memory.search(query: query, limit: limit)
120 }
121
122 func recentEvents(limit: Int = 50) async -> [MemoryEntry] {
123     await memory.recentEntries(limit: limit)
124 }
125
126 // MARK: - Private
127
128 @discardableResult
129 private func run(_ block: @escaping () async throws -> Void) async -> Bool {
130     do {
131         try await block()
132         return true
133     } catch {
134         self.lastError = error
135         #if DEBUG
136             print("[Hermes] \(error)")
137         #endif
138         return false
139     }
140 }
141 }

```

◆ HermesKairos.swift (290 lines)

```
1 import Foundation
2
3 // MARK: - Kairos ("the right time")
4 //
5 // Kairos is the always-on background observer. It watches the user's working
6 // environment, writes structured observation logs, and waits for a 15-second
7 // window of inactivity before proposing or taking autonomous action – so it
8 // never interrupts mid-thought.
9 //
10 // Three design contracts:
11 // 1. 15-Second Budget – only act after 15 s of detected user inactivity.
12 // 2. Observation Log – every environmental scan is recorded before any
13 // action is proposed, creating an audit trail.
14 // 3. Strict Write Discipline – observations are only committed to persistent
15 // memory AFTER the related action succeeds. Failed
16 // or speculative data is never stored.
17
18 // MARK: - Models
19
20 struct KairosObservation {
21     let id: UUID
22     let timestamp: Date
23     let trigger: ObservationTrigger
24     let context: [String: Any] // freeform snapshot of the environment
25     let sessionTopic: String?
26     let intent: HermesContextTracker.Intent?
27
28     enum ObservationTrigger: String {
29         case idleTimeout // 15 s of inactivity elapsed
30         case sessionStart // app foregrounded
31         case sessionEnd // app backgrounded
32         case errorDetected // tool failure logged
33         case patternRecognised // DreamEngine surfaced an insight
34     }
35 }
36
37 struct KairosAction {
38     let id: UUID
39     let observation: KairosObservation
40     let type: ActionType
41     let description: String
42     let confidence: Double // 0.0–1.0; only execute if >= threshold
43     var outcome: ActionOutcome?
44
45     enum ActionType: String {
46         case suggestContext // surface a suggestion to the UI
47         case consolidateMemory // trigger early mini-dream
48         case flagContradiction // mark conflicting memories for review
49         case promoteEntry // bump importance of a relevant entry
50         case logInsight // write a new dream_insight entry
51     }
52
53     enum ActionOutcome {
54         case success
55         case failure(Error)
56         case skipped(reason: String)
57     }
58 }
59
60 // MARK: - HermesKairos
61
62 actor HermesKairos {
63     static let shared = HermesKairos()
64
65     // Dependencies
66     private let memory = HermesMemory.shared
67     private let context = HermesContextTracker.shared
68     private let proactive = HermesProactiveEngine.shared
69
70     // State
71     private var isRunning = false
72     private var lastActivity: Date = Date()
73     private var observationLog: [KairosObservation] = []
74     private var pendingWrites: [(category: String, content: Any, metadata: [String: Any])] = []
75     // Bug fix #3: store the loop Task so pause() can cancel it and stop the chain.
```

```

76 private var loopTask: Task<Void, Never>?
77
78 // Config
79 private let idleBudgetSeconds: TimeInterval = 15
80 private let minConfidenceToAct: Double = 0.65
81 private let maxObservationLogSize = 200
82
83 private init() {}
84
85 // MARK: - Lifecycle
86
87 /// Start the Kairos background loop. Call from AppDelegate / scene activation.
88 func start() {
89     guard !isRunning else { return }
90     isRunning = true
91     scheduleNextCheck()
92 }
93
94 /// Pause Kairos (e.g. when app backgrounds – DreamEngine takes over instead).
95 func pause() {
96     isRunning = false
97     // Bug fix #3: cancel the stored task so the recursive chain terminates.
98     loopTask?.cancel()
99     loopTask = nil
100 }
101
102 /// Signal that the user is active – resets the 15-second idle budget.
103 func userDidAct() {
104     lastActivity = Date()
105 }
106
107 // MARK: - Idle check loop
108
109 private func scheduleNextCheck() {
110     // Bug fix #3: store the task so pause() can cancel it.
111     loopTask = Task { [weak self] in
112         // Poll every 5 s; fire the full observation when idle budget is met
113         try? await Task.sleep(nanoseconds: 5_000_000_000)
114         guard !Task.isCancelled, let self, await self.isRunning else { return }
115         await self.tick()
116         await self.scheduleNextCheck()
117     }
118 }
119
120 private func tick() async {
121     let idle = Date().timeIntervalSince(lastActivity)
122     guard idle >= idleBudgetSeconds else { return }
123
124     let obs = await buildObservation(trigger: .idleTimeout)
125     observationLog.append(obs)
126     trimObservationLog()
127
128     let action = await proposeAction(for: obs)
129     await executeIfWarranted(action)
130 }
131
132 // MARK: - Observation
133
134 private func buildObservation(trigger: KairosObservation.ObservationTrigger) async -> KairosObservation {
135     let topic = await context.currentTopic()
136     let intent = await context.currentIntent()
137     let recent = await memory.recentEntries(limit: 5)
138
139     let ctx: [String: Any] = [
140         "recentCategories": recent.map(\.category),
141         "idleSeconds": Date().timeIntervalSince(lastActivity),
142         "observationCount": observationLog.count
143     ]
144
145     return KairosObservation(
146         id: UUID(),
147         timestamp: Date(),
148         trigger: trigger,
149         context: ctx,
150         sessionTopic: topic,
151         intent: intent
152     )

```

```

153 }
154
155 // MARK: - Action proposal
156
157 private func proposeAction(for obs: KairosObservation) async -> KairosAction {
158 // Check for tool failures → flag or consolidate
159 let recentErrors = await memory.entries(for: "tool_exec")
160 .prefix(20)
161 .filter {
162 guard let d = $0.content.value as? [String: Any],
163 let ok = d["success"] as? Bool else { return false }
164 return !ok
165 }
166
167 if recentErrors.count >= 3 {
168 return KairosAction(
169 id: UUID(), observation: obs,
170 type: .consolidateMemory,
171 description: "Recurring failures detected – trigger early memory consolidation.",
172 confidence: 0.80
173 )
174 }
175
176 // Check if a dream insight exists that hasn't been surfaced yet
177 let unseenInsights = await memory.entries(for: "dream_insight")
178 .filter { $0.timestamp > Date().addingTimeInterval(-3600) }
179 if !unseenInsights.isEmpty {
180 return KairosAction(
181 id: UUID(), observation: obs,
182 type: .suggestContext,
183 description: "Unseen dream insight available – refresh proactive suggestions.",
184 confidence: 0.72
185 )
186 }
187
188 // Default: nothing warranted right now
189 return KairosAction(
190 id: UUID(), observation: obs,
191 type: .suggestContext,
192 description: "Idle scan complete, no urgent action.",
193 confidence: 0.10
194 )
195 }
196
197 // MARK: - Execution with strict write discipline
198
199 private func executeIfWarranted(_ action: KairosAction) async {
200 guard action.confidence >= minConfidenceToAct else {
201 recordOutcome(action, .skipped(reason: "Confidence \("\(action.confidence)\) below threshold \("\(minConfidenceToAct\)"))
202 return
203 }
204
205 var result = action
206 do {
207 try await perform(action)
208 // ■ STRICT WRITE DISCIPLINE: only flush pending writes after success
209 try await flushPendingWrites()
210 result.outcome = .success
211 } catch {
212 // ■ Do NOT write speculative data – discard pending writes
213 pendingWrites.removeAll()
214 result.outcome = .failure(error)
215 }
216 recordOutcome(result, result.outcome ?? .skipped(reason: "unknown"))
217 }
218
219 private func perform(_ action: KairosAction) async throws {
220 switch action.type {
221 case .consolidateMemory:
222 try await memory.consolidate(importanceThreshold: 2, keepWindow: 7 * 86400)
223
224 case .suggestContext:
225 await proactive.refresh()
226
227 case .promoteEntry:
228 // Promote the most recently accessed relevant entry
229 let recent = await memory.recentEntries(limit: 1)

```

```

230 if let entry = recent.first {
231     try await memory.promoteToLongTerm([entry.id])
232 }
233
234 case .logInsight:
235     let topic = action.observation.sessionTopic ?? "general"
236     // Stage the write – only committed if this action succeeds (flushPendingWrites)
237     pendingWrites.append((
238         category: "kairos_insight",
239         content: ["topic": topic, "trigger": action.observation.trigger.rawValue],
240         metadata: ["importance": 3]
241     ))
242
243 case .flagContradiction:
244     // Surfaced by DreamEngine phase 5; log for review
245     pendingWrites.append((
246         category: "kairos_flag",
247         content: ["description": action.description],
248         metadata: ["importance": 4]
249     ))
250 }
251 }
252
253 /// Flush staged writes to persistent memory – only called on success.
254 private func flushPendingWrites() async throws {
255     for write in pendingWrites {
256         try await memory.observe(category: write.category,
257             content: write.content,
258             metadata: write.metadata)
259     }
260     pendingWrites.removeAll()
261 }
262
263 private func recordOutcome(_ action: KairosAction, _ outcome: KairosAction.ActionOutcome) {
264     // Lightweight in-memory only – no disk write for outcome records
265     // (they'd create noise; the observation log is the audit trail)
266     #if DEBUG
267     let status: String
268     switch outcome {
269     case .success: status = "✓"
270     case .failure(let e): status = "✗ \(e)"
271     case .skipped(let r): status = "- \(r)"
272     }
273     print("[Kairos] \(action.type.rawValue) \(status)")
274     #endif
275 }
276
277 // MARK: - Observation log management
278
279 private func trimObservationLog() {
280     if observationLog.count > maxObservationLogSize {
281         observationLog.removeFirst(observationLog.count - maxObservationLogSize)
282     }
283 }
284
285 // MARK: - Public accessors
286
287 func recentObservations(limit: Int = 20) -> [KairosObservation] {
288     Array(observationLog.suffix(limit).reversed())
289 }
290 }

```

◆ HermesMemory.swift (414 lines)

```
1 import Foundation
2
3 // MARK: - Memory tier
4
5 /// Short-term holds raw, recent observations.
6 /// Long-term holds entries that have been promoted by the DreamEngine.
7 enum MemoryTier: String, Codable {
8     case shortTerm
9     case longTerm
10 }
11
12 // MARK: - MemoryEntry
13
14 struct MemoryEntry: Codable, Identifiable {
15     let id: UUID
16     let timestamp: Date
17     let category: String
18     let content: AnyCodable
19     let metadata: [String: AnyCodable]
20     var importance: Int // 1-5; decays over time, boosted by DreamEngine
21     var tier: MemoryTier
22     var accessCount: Int // incremented on retrieval; informs eviction
23
24     init(
25         id: UUID = UUID(),
26         timestamp: Date = Date(),
27         category: String,
28         content: Any,
29         metadata: [String: Any] = [:],
30         importance: Int = 1,
31         tier: MemoryTier = .shortTerm
32     ) {
33         self.id = id
34         self.timestamp = timestamp
35         self.category = category
36         self.content = AnyCodable(content)
37         self.metadata = metadata.mapValues { AnyCodable($0) }
38         self.importance = max(1, min(5, importance))
39         self.tier = tier
40         self.accessCount = 0
41     }
42 }
43
44 // MARK: - AnyCodable
45
46 /// Type-erased Codable wrapper for arbitrary JSON-compatible values.
47 struct AnyCodable: Codable {
48     let value: Any
49
50     init(_ value: Any) { self.value = value }
51
52     init(from decoder: Decoder) throws {
53         let c = try decoder.singleValueContainer()
54         // Bool must come before Int: in JSON, true/false are distinct from 1/0
55         if let v = try? c.decode(Bool.self) { value = v; return }
56         if let v = try? c.decode(Int.self) { value = v; return }
57         if let v = try? c.decode(Double.self) { value = v; return }
58         if let v = try? c.decode(String.self) { value = v; return }
59         if let v = try? c.decode([AnyCodable].self) { value = v.map(\.value); return }
60         if let v = try? c.decode([String: AnyCodable].self) { value = v.mapValues(\.value); return }
61         value = NSNull()
62     }
63
64     func encode(to encoder: Encoder) throws {
65         var c = encoder.singleValueContainer()
66         switch value {
67             case let v as Bool: try c.encode(v)
68             case let v as Int: try c.encode(v)
69             case let v as Double: try c.encode(v)
70             case let v as String: try c.encode(v)
71             case let v as [Any]: try c.encode(v.map { AnyCodable($0) })
72             case let v as [String: Any]: try c.encode(v.mapValues { AnyCodable($0) })
73             default: try c.encodeNil()
74         }
75 }
```



```

76 }
77
78 // MARK: - MemoryIndex (Layered Memory Architecture)
79 //
80 // A lightweight keyword → entry-ID map persisted separately from the full
81 // entry store. The HermesAgentHarness queries this index first; it only
82 // deserialises full entries for the small result set, preventing context
83 // entropy from reloading entire conversation histories.
84
85 struct MemoryIndex: Codable {
86     /// keyword → set of entry IDs that contain it
87     var keywordMap: [String: [UUID]] = [:]
88     var updatedAt: Date = Date()
89
90     /// Add an entry's keywords to the index.
91     mutating func index(entry: MemoryEntry) {
92         let words = tokenise("\(entry.content.value) \(entry.category)")
93         for word in words {
94             var ids = keywordMap[word, default: []]
95             if !ids.contains(entry.id) { ids.append(entry.id) }
96             keywordMap[word] = ids
97         }
98     }
99
100     /// Remove an entry's ID from all keyword buckets.
101     mutating func remove(id: UUID) {
102         for key in keywordMap.keys {
103             keywordMap[key]?.removeAll { $0 == id }
104         }
105     }
106
107     /// Look up entry IDs matching ALL of the given keywords (AND logic).
108     func lookup(keywords: [String]) -> Set<UUID> {
109         guard !keywords.isEmpty else { return [] }
110         let sets = keywords.map { kw -> Set<UUID> in
111             Set(keywordMap[kw.lowercased()] ?? [])
112         }
113         return sets.dropFirst().reduce(sets[0]) { $0.intersection($1) }
114     }
115
116     /// Look up entry IDs matching ANY keyword (OR logic).
117     func lookupAny(keywords: [String]) -> Set<UUID> {
118         Set(keywords.flatMap { keywordMap[$0.lowercased()] ?? [] })
119     }
120
121     private func tokenise(_ text: String) -> [String] {
122         text.lowercased()
123         .components(separatedBy: .alphanumerics.inverted)
124         .filter { $0.count > 3 }
125     }
126 }
127
128 // MARK: - HermesMemory
129
130 /// Fully on-device, sandboxed memory store.
131 ///
132 /// v3 additions:
133 /// - MemoryIndex: lightweight keyword map persisted separately for O(1) search
134 /// without deserialising full entry payloads (layered memory architecture)
135 actor HermesMemory {
136     static let shared = HermesMemory()
137 }
138 // MARK: - Constants
139
140 private let shortTermCap = 2_000
141 private let decayIntervalDays: Double = 3
142 private let persistDebounceSeconds: Double = 2.0
143
144 // MARK: - Storage
145
146 private var entries: [MemoryEntry] = []
147 /// Category → entry IDs (maintained in sync with `entries`)
148 private var categoryIndex: [String: [UUID]] = [:]
149 /// Lightweight keyword index (separate file, fast to load)
150 private var memoryIndex: MemoryIndex = MemoryIndex()
151 private var loaded = false
152

```

```

153 /// Pending debounced persist task
154 private var pendingPersistTask: Task<Void, Never>?
155
156 // MARK: - Paths
157
158 private let memoryDir: URL = {
159     FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]
160     .appendingPathComponent("hermes", isDirectory: true)
161 }()
162
163 private var shortTermFile: URL { memoryDir.appendingPathComponent("short_term.json") }
164 private var longTermFile: URL { memoryDir.appendingPathComponent("long_term.json") }
165 private var indexFile: URL { memoryDir.appendingPathComponent("memory_index.json") }
166
167 // MARK: - Encoder / Decoder
168
169 private let encoder: JSONEncoder = {
170     let e = JSONEncoder()
171     e.dateEncodingStrategy = .iso8601
172     e.outputFormatting = .prettyPrinted
173     return e
174 }()
175
176 private let decoder: JSONDecoder = {
177     let d = JSONDecoder()
178     d.dateDecodingStrategy = .iso8601
179     return d
180 }()
181
182 private init() {}
183
184 // MARK: - Write
185
186 /// Record a new observation. Returns the created entry's ID.
187 @discardableResult
188 func observe(category: String, content: Any, metadata: [String: Any] = [:]) async throws -> UUID {
189     try ensureDir()
190     await loadIfNeeded()
191
192     let importance = (metadata["importance"] as? Int) ?? 1
193     let entry = MemoryEntry(category: category, content: content,
194         metadata: metadata, importance: importance)
195     append(entry)
196     applyDecay()
197     evictIfNeeded()
198     schedulePersist()
199     return entry.id
200 }
201
202 /// Batch-update multiple entries in one persist round-trip.
203 func updateBatch(_ updated: [MemoryEntry]) async throws {
204     await loadIfNeeded()
205     for entry in updated {
206         if let idx = entries.firstIndex(where: { $0.id == entry.id }) {
207             entries[idx] = entry
208         }
209     }
210     try await persistNow()
211 }
212
213 /// Update a single entry.
214 func update(_ entry: MemoryEntry) async throws {
215     try await updateBatch([entry])
216 }
217
218 // MARK: - Read
219
220 /// All entries, newest first.
221 func allEntries() async -> [MemoryEntry] {
222     await loadIfNeeded()
223     return entries.reversed()
224 }
225
226 /// Most recent N entries, newest first.
227 func recentEntries(limit: Int = 50) async -> [MemoryEntry] {
228     await loadIfNeeded()
229     return Array(entries.suffix(limit).reversed())

```

```

230 }
231
232 /// Entries in a given category, newest first – 0(index) lookup.
233 func entries(for category: String) async -> [MemoryEntry] {
234     await loadIfNeeded()
235     let ids = Set(categoryIndex[category] ?? [])
236     return entries
237     .filter { ids.contains($0.id) }
238     .reversed()
239 }
240
241 /// Entries matching multiple categories.
242 func entries(forAny categories: [String]) async -> [MemoryEntry] {
243     await loadIfNeeded()
244     let ids = categories.flatMap { categoryIndex[$0] ?? [] }
245     let idSet = Set(ids)
246     return entries.filter { idSet.contains($0.id) }.reversed()
247 }
248
249 /// Keyword search across serialised content and category name.
250 func search(query: String, limit: Int = 20) async -> [MemoryEntry] {
251     await loadIfNeeded()
252     let q = query.lowercased()
253     let matches = entries
254     .filter { "\($0.content.value)".lowercased().contains(q)
255     || $0.category.lowercased().contains(q) }
256     // Bump access count for found entries
257     let ids = Set(matches.map(\.id))
258     for i in entries.indices where ids.contains(entries[i].id) {
259         entries[i].accessCount += 1
260     }
261     return Array(matches.suffix(limit).reversed())
262 }
263
264 // MARK: - Maintenance
265
266 /// Prune low-importance, old short-term entries (called by DreamEngine).
267 func consolidate(importanceThreshold: Int = 2, keepWindow: TimeInterval = 7 * 86400) async throws {
268     await loadIfNeeded()
269     let cutoff = Date().addingTimeInterval(-keepWindow)
270     let before = entries.count
271     entries = entries.filter {
272         $0.tier == .longTerm
273         || $0.importance >= importanceThreshold
274         || $0.timestamp > cutoff
275     }
276     rebuildCategoryIndex()
277     if entries.count != before { try await persistNow() }
278 }
279
280 /// Promote entries to long-term tier.
281 func promoteToLongTerm(_ ids: [UUID]) async throws {
282     await loadIfNeeded()
283     var changed = false
284     for i in entries.indices where ids.contains(entries[i].id) {
285         if entries[i].tier != .longTerm {
286             entries[i].tier = .longTerm
287             changed = true
288         }
289     }
290     if changed { try await persistNow() }
291 }
292
293 // MARK: - Private helpers
294
295 private func append(_ entry: MemoryEntry) {
296     entries.append(entry)
297     categoryIndex[entry.category, default: []].append(entry.id)
298     memoryIndex.index(entry: entry)
299 }
300
301 private func rebuildCategoryIndex() {
302     categoryIndex = [:]
303     memoryIndex = MemoryIndex()
304     for entry in entries {
305         categoryIndex[entry.category, default: []].append(entry.id)
306         memoryIndex.index(entry: entry)
307     }
308 }

```

```

307 }
308 }
309
310 // MARK: - Index-based search (layered architecture: no full deserialisation)
311
312 /// Fast keyword search via MemoryIndex – resolves only matching entry IDs.
313 func indexSearch(keywords: [String], limit: Int = 20) async -> [MemoryEntry] {
314     await loadIfNeeded()
315     let ids = memoryIndex.lookupAny(keywords: keywords.map { $0.lowercased() })
316     let idMap = Dictionary(uniqueKeysWithValues: entries.map { ($0.id, $0) })
317     return ids
318         .compactMap { idMap[$0] }
319         .sorted { $0.timestamp > $1.timestamp }
320         .prefix(limit)
321         .map { $0 }
322 }
323
324 /// Importance decay: short-term entries lose 1 point per `decayIntervalDays` days of age.
325 private func applyDecay() {
326     let now = Date()
327     for i in entries.indices where entries[i].tier == .shortTerm {
328         let ageDays = now.timeIntervalSince(entries[i].timestamp) / 86400
329         let decaySteps = Int(ageDays / decayIntervalDays)
330         let decayed = max(1, entries[i].importance - decaySteps)
331         if decayed != entries[i].importance {
332             entries[i].importance = decayed
333         }
334     }
335 }
336
337 /// Evict short-term entries beyond cap: lowest importance first, then oldest.
338 private func evictIfNeeded() {
339     // Bug fix #6: count only shortTerm entries against the cap – longTerm
340     // entries are permanently retained and must not trigger eviction.
341     let shortTerm = entries.filter { $0.tier == .shortTerm }
342     guard shortTerm.count > shortTermCap else { return }
343
344     let overflow = shortTerm.count - shortTermCap
345     let toEvict = Set(
346         shortTerm
347         .sorted { lhs, rhs in
348             if lhs.importance != rhs.importance { return lhs.importance < rhs.importance }
349             return lhs.timestamp < rhs.timestamp
350         }
351         .prefix(overflow)
352         .map { $0.id }
353     )
354     entries.removeAll { toEvict.contains($0.id) }
355     rebuildCategoryIndex()
356 }
357
358 /// Debounced: cancels any pending write and schedules a new one 2 s out.
359 private func schedulePersist() {
360     pendingPersistTask?.cancel()
361     pendingPersistTask = Task { [weak self] in
362         try? await Task.sleep(nanoseconds: UInt64(2_000_000_000))
363         guard !Task.isCancelled, let self else { return }
364         try? await self.persistNow()
365     }
366 }
367
368 // MARK: - Persistence
369
370 private func loadIfNeeded() async {
371     guard !loaded else { return }
372     loaded = true
373     // Load both tiers and merge
374     var all: [MemoryEntry] = []
375     for file in [shortTermFile, longTermFile] {
376         if let data = try? Data(contentsOf: file),
377         let decoded = try? decoder.decode([MemoryEntry].self, from: data) {
378             all.append(contentsOf: decoded)
379         }
380     }
381     entries = all.sorted { $0.timestamp < $1.timestamp }
382 }
383 // Load the lightweight index if present; rebuild if missing or stale

```

```

384 if let idxData = try? Data(contentsOf: indexFile),
385 let idx = try? decoder.decode(MemoryIndex.self, from: idxData) {
386     memoryIndex = idx
387     // Rebuild category index from entries (cheap)
388     categoryIndex = [:]
389     for entry in entries {
390         categoryIndex[entry.category, default: []].append(entry.id)
391     }
392 } else {
393     rebuildCategoryIndex() // also rebuilds memoryIndex
394 }
395 }
396
397 func persistNow() async throws {
398     pendingPersistTask?.cancel()
399     pendingPersistTask = nil
400     let short = entries.filter { $0.tier == .shortTerm }
401     let long = entries.filter { $0.tier == .longTerm }
402     try encoder.encode(short).write(to: shortTermFile, options: .atomic)
403     if !long.isEmpty {
404         try encoder.encode(long).write(to: longTermFile, options: .atomic)
405     }
406     // Persist the lightweight index separately (much smaller than full entry files)
407     let idxData = try encoder.encode(memoryIndex)
408     try idxData.write(to: indexFile, options: .atomic)
409 }
410
411 private func ensureDir() throws {
412     try FileManager.default.createDirectory(at: memoryDir, withIntermediateDirectories: true)
413 }
414 }

```

◆ HermesProactiveEngine.swift (236 lines)

```
1 import Foundation
2
3 // MARK: - HermesSuggestion
4
5 struct HermesSuggestion: Identifiable {
6     let id: UUID
7     let title: String
8     let body: String
9     let category: SuggestionCategory
10    let confidence: Double // 0.0-1.0
11    let createdAt: Date
12    /// Stable key used for deduplication across evaluations.
13    let dedupeKey: String
14
15    enum SuggestionCategory: String {
16        case reminder // "Pick up where you left off?"
17        case caution // "That tool keeps failing."
18        case insight // "While you were away, I noticed..."
19        case encouragement // "You're on a roll today."
20    }
21 }
22
23 // MARK: - HermesProactiveEngine
24 //
25 // Always-on engine that surfaces friendly, context-aware suggestions.
26 //
27 // Improvements over v1:
28 // - Removed unused `import Combine`
29 // - Suggestion deduplication: same dedupe key won't surface twice in the same session
30 // - Smarter interrupted-work detection: checks session gap (> 30 min since last msg),
31 //   not a fragile 120 s window
32 // - `inout [HermesSuggestion]` helpers replaced with returning arrays to avoid
33 //   async+inout complexity
34
35 actor HermesProactiveEngine {
36     static let shared = HermesProactiveEngine()
37
38     private let memory = HermesMemory.shared
39     private var cachedSuggestions: [HermesSuggestion] = []
40     private var lastEvaluated: Date = .distantPast
41
42     /// Suggestion dedupe keys seen since app launch (reset on cold start only).
43     private var seenDedupeKeys: Set<String> = []
44
45     private let minEvalInterval: TimeInterval = 60
46
47     private init() {}
48
49     // MARK: - Public API
50
51     func currentSuggestions() async -> [HermesSuggestion] {
52         if Date().timeIntervalSince(lastEvaluated) > minEvalInterval {
53             await evaluate()
54         }
55         return cachedSuggestions
56     }
57
58     func refresh() async {
59         await evaluate()
60     }
61
62     // MARK: - Evaluation
63
64     private func evaluate() async {
65         lastEvaluated = Date()
66
67         var candidates: [HermesSuggestion] = []
68         candidates += await checkUnfinishedWork()
69         candidates += await checkRepeatedFailures()
70         candidates += await checkDreamInsights()
71         candidates += await checkEncouragement()
72
73         // Deduplicate: skip any suggestion whose key was already shown this session
74         let fresh = candidates.filter { !seenDedupeKeys.contains($0.dedupeKey) }
75     }
```

```

76 // Top 5 by confidence
77 let top = fresh.sorted { $0.confidence > $1.confidence }.prefix(5).map { $0 }
78
79 // Record shown keys so they don't repeat
80 top.forEach { seenDedupeKeys.insert($0.dedupeKey) }
81
82 cachedSuggestions = top
83 }
84
85 // MARK: - Checks
86
87 /// Fires when the last user message has no assistant response AND was sent > 30 min ago,
88 /// suggesting the session ended before the answer arrived.
89 private func checkUnfinishedWork() async -> [HermesSuggestion] {
90 let messages = await memory.entries(for: "user_message")
91 let responses = await memory.entries(for: "assistant_response")
92 guard let lastMsg = messages.first else { return [] } // entries() returns newest first
93
94 let sessionGap: TimeInterval = 30 * 60
95 let isOld = Date().timeIntervalSince(lastMsg.timestamp) > sessionGap
96
97 guard isOld else { return [] }
98
99 let hasResponse = responses.contains {
100 $0.timestamp > lastMsg.timestamp
101 }
102 guard !hasResponse else { return [] }
103
104 let topic = (lastMsg.content.value as? [String: Any])?["text"] as? String ?? "your last question"
105 let preview = String(topic.prefix(72))
106
107 return [HermesSuggestion(
108 id: UUID(),
109 title: "Pick up where you left off?",
110 body: "Looks like we got cut off. You were asking: \"\$(preview)...\" – want to continue that?",
111 category: .reminder,
112 confidence: 0.85,
113 createdAt: Date(),
114 dedupeKey: "unfinished_\$(lastMsg.id)"
115 )]
116 }
117
118 private func checkRepeatedFailures() async -> [HermesSuggestion] {
119 let execs = await memory.entries(for: "tool_exec")
120 let recent = execs.filter { $0.timestamp > Date().addingTimeInterval(-3600) }
121
122 let failures: [String] = recent.compactMap {
123 guard let d = $0.content.value as? [String: Any],
124 let ok = d["success"] as? Bool, !ok,
125 let name = d["tool"] as? String else { return nil }
126 return name
127 }
128
129 let counts = Dictionary(failures.map { ($0, 1) }, uniquingKeysWith: +)
130 return counts.compactMap { tool, count -> HermesSuggestion? in
131 guard count >= 3 else { return nil }
132 return HermesSuggestion(
133 id: UUID(),
134 title: "\"\$(tool)' keeps failing",
135 body: "That tool has hit an error \$(count) times in the last hour. It might need different input or t...
136 category: .caution,
137 confidence: min(0.5 + Double(count) * 0.1, 0.95),
138 createdAt: Date(),
139 dedupeKey: "failure_\$(tool)"
140 )
141 }
142 }
143
144 private func checkDreamInsights() async -> [HermesSuggestion] {
145 var out: [HermesSuggestion] = []
146
147 let insights = await memory.entries(for: "dream_insight")
148 .filter { $0.timestamp > Date().addingTimeInterval(-86400) }
149
150 for insight in insights.prefix(2) {
151 guard let text = (insight.content.value as? [String: Any])?["insight"] as? String,
152 !text.isEmpty else { continue }

```

```

153 out.append(HermesSuggestion(
154     id: UUID(),
155     title: "Something I noticed while you were away",
156     body: text + " Let me know if you'd like to dive into any of these.",
157     category: .insight,
158     confidence: 0.70,
159     createdAt: insight.timestamp,
160     dedupeKey: "insight_\(insight.id)"
161 ))
162 }
163
164 let improvements = await memory.entries(for: "self_improvement")
165 .filter { $0.timestamp > Date().addingTimeInterval(-86400) }
166
167 for imp in improvements.prefix(1) {
168     guard let note = (imp.content.value as? [String: Any])?["note"] as? String,
169     !note.isEmpty else { continue }
170     out.append(HermesSuggestion(
171         id: UUID(),
172         title: "Quick heads-up from last night",
173         body: note,
174         category: .caution,
175         confidence: 0.80,
176         createdAt: imp.timestamp,
177         dedupeKey: "improvement_\(imp.id)"
178     ))
179 }
180
181 return out
182 }
183
184 private func checkEncouragement() async -> [HermesSuggestion] {
185     let todayStart = Calendar.current.startOfDay(for: Date())
186     let todayMessages = await memory.entries(for: "user_message")
187     .filter { $0.timestamp >= todayStart }
188
189     guard todayMessages.count >= 10 else { return [] }
190
191     // Don't re-encourage today
192     let alreadyEncouraged = await memory.entries(for: "encouragement_sent")
193     .contains { $0.timestamp >= todayStart }
194     guard !alreadyEncouraged else { return [] }
195
196     Task {
197         try? await memory.observe(
198             category: "encouragement_sent",
199             content: ["count": todayMessages.count],
200             metadata: ["importance": 1]
201         )
202     }
203
204     return [HermesSuggestion(
205         id: UUID(),
206         title: "You're on a roll today",
207         body: "You've sent \(todayMessages.count) messages today – that's a productive session. Remember to take ...
208         category: .encouragement,
209         confidence: 0.60,
210         createdAt: Date(),
211         dedupeKey: "encouragement_\(todayStart.timeIntervalSince1970)"
212     )]
213 }
214 }
215
216 // MARK: - SwiftUI ViewModel
217
218 @MainActor
219 final class HermesViewModel: ObservableObject {
220     @Published var suggestions: [HermesSuggestion] = []
221     @Published var isLoading = false
222
223     private var refreshTask: Task<Void, Never>?
224
225     func refresh() {
226         // Cancel any in-flight refresh before starting a new one
227         refreshTask?.cancel()
228         isLoading = true
229         refreshTask = Task { [weak self] in

```


◆ HermesSessionState.swift (283 lines)

```
1 import Foundation
2
3 // MARK: - HermesSessionState
4 //
5 // Crash-safe session state: every meaningful change is persisted atomically
6 // before the operation completes, so a crash or force-quit never loses data.
7 //
8 // Three layers:
9 // ConversationState – message history, last cursor, unread count
10 // WorkflowState – long-running task state separate from chat history
11 // (can be retried safely; survives crashes)
12 // TokenBudget – hard limits with pre-turn checks; halts execution
13 // before spending over-budget
14
15 // MARK: - ConversationState
16
17 struct ConversationState: Codable {
18     var id: String
19     var messageCount: Int
20     var lastMessageTimestamp: Date?
21     var lastAssistantCursor: String? // opaque resume token (e.g. API message ID)
22     var permissionTier: PermissionTier
23
24     static func fresh(id: String = UUID().uuidString) -> ConversationState {
25         ConversationState(
26             id: id, messageCount: 0,
27             lastMessageTimestamp: nil,
28             lastAssistantCursor: nil,
29             permissionTier: .standard
30         )
31     }
32 }
33
34 // MARK: - WorkflowState
35
36 /// Long-running task state kept separate from chat history.
37 /// Designed for safe retry: re-running a workflow from its saved state
38 /// must be idempotent.
39 struct WorkflowState: Codable {
40     enum Status: String, Codable {
41         case idle, running, paused, completed, failed
42     }
43
44     var workflowId: String
45     var status: Status
46     var currentStepIndex: Int
47     var totalSteps: Int
48     var stepCheckpoints: [String: AnyCodable] // step name → last known result
49     var failureReason: String?
50     var startedAt: Date?
51     var updatedAt: Date
52
53     static func idle() -> WorkflowState {
54         WorkflowState(
55             workflowId: UUID().uuidString,
56             status: .idle,
57             currentStepIndex: 0,
58             totalSteps: 0,
59             stepCheckpoints: [:],
60             failureReason: nil,
61             startedAt: nil,
62             updatedAt: Date()
63         )
64     }
65
66     /// Advance to next step, persisting the result of the current one.
67     mutating func advanceStep(name: String, result: Any) {
68         stepCheckpoints[name] = AnyCodable(result)
69         currentStepIndex += 1
70         updatedAt = Date()
71     }
72
73     // Bug fix #5: only mark completed when totalSteps > 0 AND we've
74     // reached the end – prevents instant completion when totalSteps is 0.
75     if totalSteps > 0 {
76         status = currentStepIndex >= totalSteps ? .completed : .running
77     }
78 }
```

```

76 }
77 }
78
79 // MARK: - TokenBudget
80
81 struct TokenBudget: Codable {
82     var sessionLimit: Int // hard cap for this session
83     var turnLimit: Int // max tokens per single turn
84     var used: Int // running total this session
85     var lastTurnUsed: Int // tokens used in the most recent turn
86
87     var remaining: Int { max(0, sessionLimit - used) }
88     var remainingThisTurn: Int { max(0, turnLimit - lastTurnUsed) }
89     var isExhausted: Bool { used >= sessionLimit }
90     var usagePercent: Double { Double(used) / Double(sessionLimit) }
91
92     /// Pre-turn check. Returns false if there is not enough budget to start a turn.
93     func canStartTurn(estimatedCost: Int = 500) -> Bool {
94         !isExhausted && remaining >= estimatedCost
95     }
96
97     mutating func recordTurn(prompt: Int, completion: Int) {
98         let total = prompt + completion
99         used += total
100         lastTurnUsed = total
101     }
102
103     mutating func resetTurnCounter() {
104         lastTurnUsed = 0
105     }
106
107     static func `default`() -> TokenBudget {
108         TokenBudget(sessionLimit: 100_000, turnLimit: 8_000, used: 0, lastTurnUsed: 0)
109     }
110
111     static func frugal() -> TokenBudget {
112         TokenBudget(sessionLimit: 20_000, turnLimit: 2_000, used: 0, lastTurnUsed: 0)
113     }
114 }
115
116 // MARK: - Full session snapshot
117
118 struct SessionSnapshot: Codable {
119     var conversation: ConversationState
120     var workflow: WorkflowState
121     var tokenBudget: TokenBudget
122     var savedAt: Date
123     var appVersion: String
124     var schemaVersion: Int = 1
125 }
126
127 // MARK: - HermesSessionState actor
128
129 actor HermesSessionState {
130     static let shared = HermesSessionState()
131
132     private var snapshot: SessionSnapshot
133     private let encoder: JSONEncoder
134     private let decoder: JSONDecoder
135
136     private let saveFile: URL = {
137         FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]
138             .appendingPathComponent("hermes/session_state.json")
139     }()
140
141     private init() {
142         encoder = JSONEncoder()
143         encoder.dateEncodingStrategy = .iso8601
144         encoder.outputFormatting = .prettyPrinted
145         decoder = JSONDecoder()
146         decoder.dateDecodingStrategy = .iso8601
147         snapshot = SessionSnapshot(
148             conversation: .fresh(),
149             workflow: .idle(),
150             tokenBudget: .default(),
151             savedAt: Date(),
152             appVersion: Bundle.main.shortVersion

```

```

153 )
154 // Bug fix #4: load persisted state immediately on init so callers
155 // never accidentally start with a blank session after a crash.
156 Task { await self.loadFromDisk() }
157 }
158
159 // MARK: - Lifecycle
160
161 /// Restore previous session from disk (call at app launch).
162 func loadFromDisk() async {
163     guard let data = try? Data(contentsOf: saveFile),
164     let saved = try? decoder.decode(SessionSnapshot.self, from: data) else { return }
165     snapshot = saved
166 }
167
168 /// Persist current snapshot atomically (crash-safe).
169 func saveToDisk() async throws {
170     snapshot.savedAt = Date()
171     let data = try encoder.encode(snapshot)
172     try FileManager.default.createDirectory(
173     at: saveFile.deletingLastPathComponent(),
174     withIntermediateDirectories: true
175 )
176     try data.write(to: saveFile, options: .atomic)
177 }
178
179 // MARK: - Conversation
180
181 func startConversation(id: String) async throws {
182     snapshot.conversation = .fresh(id: id)
183     snapshot.workflow = .idle()
184     snapshot.tokenBudget.resetTurnCounter()
185     try await saveToDisk()
186 }
187
188 func recordMessage() async throws {
189     snapshot.conversation.messageCount += 1
190     snapshot.conversation.lastMessageTimestamp = Date()
191     try await saveToDisk()
192 }
193
194 func updateCursor(_ cursor: String) async throws {
195     snapshot.conversation.lastAssistantCursor = cursor
196     try await saveToDisk()
197 }
198
199 // MARK: - Workflow
200
201 func beginWorkflow(steps: Int) async throws {
202     snapshot.workflow = WorkflowState(
203     workflowId: UUID().uuidString,
204     status: .running,
205     currentStepIndex: 0,
206     totalSteps: steps,
207     stepCheckpoints: [:],
208     failureReason: nil,
209     startedAt: Date(),
210     updatedAt: Date()
211 )
212     try await saveToDisk()
213 }
214
215 func advanceWorkflowStep(name: String, result: Any) async throws {
216     // Bug fix #5: status transition is now handled inside advanceStep itself.
217     snapshot.workflow.advanceStep(name: name, result: result)
218     try await saveToDisk()
219 }
220
221 func failWorkflow(reason: String) async throws {
222     snapshot.workflow.status = .failed
223     snapshot.workflow.failureReason = reason
224     snapshot.workflow.updatedAt = Date()
225     try await saveToDisk()
226 }
227
228 // MARK: - Token budgeting
229

```

```

230 /// Pre-turn budget check. Throws if budget is insufficient.
231 func checkBudget(estimatedCost: Int = 500) throws {
232     guard snapshot.tokenBudget.canStartTurn(estimatedCost: estimatedCost) else {
233         throw SessionError.tokenBudgetExhausted(
234             used: snapshot.tokenBudget.used,
235             limit: snapshot.tokenBudget.sessionLimit
236         )
237     }
238 }
239
240 func recordTokenUsage(prompt: Int, completion: Int) async throws {
241     snapshot.tokenBudget.recordTurn(prompt: prompt, completion: completion)
242     try await saveToDisk()
243 }
244
245 func setBudget(_ budget: TokenBudget) async throws {
246     snapshot.tokenBudget = budget
247     try await saveToDisk()
248 }
249
250 // MARK: - Accessors
251
252 var conversation: ConversationState { snapshot.conversation }
253 var workflow: WorkflowState { snapshot.workflow }
254 var tokenBudget: TokenBudget { snapshot.tokenBudget }
255 var currentSnapshot: SessionSnapshot { snapshot }
256 }
257
258 // MARK: - Errors
259
260 enum SessionError: Error, LocalizedError {
261     case tokenBudgetExhausted(used: Int, limit: Int)
262     case workflowNotRunning
263     case snapshotCorrupted
264
265     var errorDescription: String? {
266         switch self {
267             case .tokenBudgetExhausted(let used, let limit):
268                 return "Token budget exhausted: \(used)/\(limit) used. Start a new session to continue."
269             case .workflowNotRunning:
270                 return "No active workflow to advance."
271             case .snapshotCorrupted:
272                 return "Session snapshot could not be decoded. Starting fresh."
273         }
274     }
275 }
276
277 // MARK: - Bundle helper
278
279 private extension Bundle {
280     var shortVersion: String {
281         infoDictionary?["CFBundleShortVersionString"] as? String ?? "0.0"
282     }
283 }

```

◆ HermesTheme.swift (123 lines)

```
1 import SwiftUI
2
3 // MARK: - OpenClaw Theme
4 //
5 // Dark, focused aesthetic inspired by a terminal / IDE feel.
6 // Primary brand colour: Electric Blue (#0A84FF → system)
7 // Accent: Amber Claw (#FF9F0A)
8 // Background: Midnight Navy (#0D1117)
9 // Surface: Deep Slate (#161B22)
10 // Border: Ghost Rail (#30363D)
11
12 // MARK: - Colour palette
13
14 extension Color {
15     enum OC {
16         // Backgrounds
17         static let background = Color(hex: "#0D1117") // Midnight Navy
18         static let surface = Color(hex: "#161B22") // Deep Slate
19         static let surfaceRaised = Color(hex: "#1C2128") // Card / sheet layer
20         static let border = Color(hex: "#30363D") // Ghost Rail
21
22         // Brand
23         static let primary = Color(hex: "#2F81F7") // Electric Blue
24         static let accent = Color(hex: "#FF9F0A") // Amber Claw
25         static let accentSoft = Color(hex: "#FF9F0A").opacity(0.15)
26
27         // Text
28         static let textPrimary = Color(hex: "#E6EDF3")
29         static let textSecondary = Color(hex: "#8B949E")
30         static let textMuted = Color(hex: "#484F58")
31
32         // Semantic
33         static let success = Color(hex: "#3FB950")
34         static let warning = Color(hex: "#D29922")
35         static let danger = Color(hex: "#F85149")
36         static let info = Color(hex: "#58A6FF")
37
38         // Dream mode highlight
39         static let dreamPurple = Color(hex: "#BC8CFF")
40         static let dreamPurpleSoft = Color(hex: "#BC8CFF").opacity(0.12)
41     }
42 }
43
44 // MARK: - Typography
45
46 enum OCFont {
47     static func title(_ size: CGFloat = 22) -> Font {
48         .system(size: size, weight: .bold, design: .rounded)
49     }
50     static func headline(_ size: CGFloat = 16) -> Font {
51         .system(size: size, weight: .semibold, design: .rounded)
52     }
53     static func body(_ size: CGFloat = 14) -> Font {
54         .system(size: size, weight: .regular, design: .default)
55     }
56     static func caption(_ size: CGFloat = 12) -> Font {
57         .system(size: size, weight: .regular, design: .monospaced)
58     }
59     static func mono(_ size: CGFloat = 13) -> Font {
60         .system(size: size, weight: .regular, design: .monospaced)
61     }
62 }
63
64 // MARK: - Spacing & radius
65
66 enum OCSizing {
67     static let radiusSM: CGFloat = 6
68     static let radiusMD: CGFloat = 10
69     static let radiusLG: CGFloat = 16
70     static let radiusXL: CGFloat = 24
71
72     static let spacingXS: CGFloat = 4
73     static let spacingSM: CGFloat = 8
74     static let spacingMD: CGFloat = 16
75     static let spacingLG: CGFloat = 24
```

```

76 static let spacingXL: CGFloat = 36
77 }
78
79 // MARK: - Common view modifiers
80
81 struct OCCardStyle: ViewModifier {
82 func body(content: Content) -> some View {
83 content
84 .background(Color.OC.surfaceRaised)
85 .cornerRadius(OC sizing.radiusMD)
86 .overlay(
87 RoundedRectangle(cornerRadius: OC sizing.radiusMD)
88 .strokeBorder(Color.OC.border, lineWidth: 1)
89 )
90 }
91 }
92
93 struct OCAccentBadge: ViewModifier {
94 let color: Color
95 func body(content: Content) -> some View {
96 content
97 .padding(.horizontal, OC sizing.spacingSM)
98 .padding(.vertical, OC sizing.spacingXS)
99 .background(color.opacity(0.15))
100 .foregroundColor(color)
101 .cornerRadius(OC sizing.radiusSM)
102 .font(OCFont.caption())
103 }
104 }
105
106 extension View {
107 func ocCard() -> some View { modifier(OCCardStyle()) }
108 func ocBadge(_ color: Color = .OC.accent) -> some View { modifier(OCAccentBadge(color: color)) }
109 }
110
111 // MARK: - Hex colour convenience
112
113 extension Color {
114 init(hex: String) {
115 let h = hex.trimmingCharacters(in: CharacterSet.alphanumerics.inverted)
116 var int: UInt64 = 0
117 Scanner(string: h).scanHexInt64(&int)
118 let r = Double((int >> 16) & 0xFF) / 255
119 let g = Double((int >> 8) & 0xFF) / 255
120 let b = Double(int & 0xFF) / 255
121 self.init(red: r, green: g, blue: b)
122 }
123 }

```

◆ HermesToolRegistry.swift (312 lines)

```
1 import Foundation
2
3 // MARK: - Tool Registry
4 //
5 // Metadata-first: every tool is described (name, capabilities, required permission
6 // tier) before any execution path is defined. The harness consults this registry
7 // to assemble a context-appropriate tool pool and run permission checks.
8 //
9 // Permission tiers (ascending trust):
10 // readonly – read-only data access, no side-effects
11 // standard – normal app operations (writing memory, UI updates)
12 // privileged – sensitive operations (network, file writes, session data)
13 // restricted – dangerous / destructive (only callable in explicit harness mode)
14
15 // MARK: - Permission Tier
16
17 enum PermissionTier: Int, Comparable, Codable {
18     case readonly = 0
19     case standard = 1
20     case privileged = 2
21     case restricted = 3
22
23     static func < (lhs: PermissionTier, rhs: PermissionTier) -> Bool {
24         lhs.rawValue < rhs.rawValue
25     }
26 }
27
28 // MARK: - Tool Capability flags
29
30 struct ToolCapabilities: OptionSet, Codable {
31     let rawValue: UInt16
32
33     static let readMemory = ToolCapabilities(rawValue: 1 << 0)
34     static let writeMemory = ToolCapabilities(rawValue: 1 << 1)
35     static let readSession = ToolCapabilities(rawValue: 1 << 2)
36     static let writeSession = ToolCapabilities(rawValue: 1 << 3)
37     static let modifyUI = ToolCapabilities(rawValue: 1 << 4)
38     static let readContext = ToolCapabilities(rawValue: 1 << 5)
39     static let triggerDream = ToolCapabilities(rawValue: 1 << 6)
40     static let networkAccess = ToolCapabilities(rawValue: 1 << 7)
41     static let fileWrite = ToolCapabilities(rawValue: 1 << 8)
42     static let executeSubagent = ToolCapabilities(rawValue: 1 << 9)
43
44     /// Convenience: all safe read-only capabilities
45     static let readonly: ToolCapabilities = [.readMemory, .readSession, .readContext]
46 }
47
48 // MARK: - Security module checks (18 modules)
49 //
50 // Each SecurityModule is a lightweight predicate evaluated before a privileged
51 // or restricted tool executes. If any required module returns false, execution
52 // is denied and the reason is logged.
53
54 struct SecurityModule {
55     let name: String
56     let check: (ToolDefinition, PermissionContext) -> Bool
57 }
58
59 struct PermissionContext {
60     let sessionTier: PermissionTier // highest tier granted for this session
61     let isInForeground: Bool
62     let tokenBudgetRemaining: Int // prevents runaway tool chains
63     let kairosActive: Bool
64     let metadata: [String: Any]
65 }
66
67 // MARK: - ToolDefinition
68
69 struct ToolDefinition: Identifiable {
70     let id: String // stable, reverse-domain identifier
71     let displayName: String
72     let description: String
73     let capabilities: ToolCapabilities
74     let requiredTier: PermissionTier
75     let agentRoles: [AgentRole] // which agent roles can call this tool
```



```

76 let inputSchema: [String: Any] // JSON Schema describing expected input
77 var isEnabled: Bool = true
78
79 /// Tags used by Tool Pool Assembly to select context-appropriate subsets.
80 let contextTags: Set<String>
81 }
82
83 // MARK: - HermesToolRegistry
84
85 actor HermesToolRegistry {
86 static let shared = HermesToolRegistry()
87
88 private var tools: [String: ToolDefinition] = [:]
89 private var securityModules: [SecurityModule] = []
90
91 private init() {
92 bootstrapBuiltinTools()
93 bootstrapSecurityModules()
94 }
95
96 // MARK: - Registration
97
98 func register(_ tool: ToolDefinition) {
99 tools[tool.id] = tool
100 }
101
102 func disable(_ id: String) {
103 tools[id]?.isEnabled = false
104 }
105
106 // MARK: - Tool Pool Assembly
107
108 /// Returns enabled tools appropriate for the given agent role and context tags.
109 func assemblePool(for role: AgentRole, contextTags: Set<String> = []) -> [ToolDefinition] {
110 tools.values
111 .filter { $0.isEnabled }
112 .filter { $0.agentRoles.contains(role) }
113 .filter { contextTags.isEmpty || !$0.contextTags.isDisjoint(with: contextTags) }
114 .sorted { $0.id < $1.id }
115 }
116
117 /// All tools available at or below the given permission tier.
118 func tools(upTo tier: PermissionTier) -> [ToolDefinition] {
119 tools.values.filter { $0.isEnabled && $0.requiredTier <= tier }.sorted { $0.id < $1.id }
120 }
121
122 // MARK: - Permission check
123
124 /// Run all required security modules. Returns .success or the first denial reason.
125 func validate(tool id: String, context: PermissionContext) -> PermissionResult {
126 guard let tool = tools[id] else {
127 return .denied("Tool '\(id)' not registered.")
128 }
129 guard tool.isEnabled else {
130 return .denied("Tool '\(id)' is disabled.")
131 }
132 guard context.sessionTier >= tool.requiredTier else {
133 return .denied("Session tier \((context.sessionTier) insufficient for \((tool.requiredTier) tool.")
134 }
135 guard context.tokenBudgetRemaining > 0 else {
136 return .denied("Token budget exhausted.")
137 }
138
139 // Run all security modules in order
140 for module in securityModules {
141 if !module.check(tool, context) {
142 return .denied("Security module '\(module.name)' rejected execution.")
143 }
144 }
145 return .allowed
146 }
147
148 enum PermissionResult {
149 case allowed
150 case denied(String)
151 var isAllowed: Bool { if case .allowed = self { return true }; return false }
152 }

```

```

153
154 // MARK: - Lookup
155
156 func tool(_ id: String) -> ToolDefinition? { tools[id] }
157 var allTools: [ToolDefinition] { Array(tools.values) }
158
159 // MARK: - Built-in tool bootstrap
160
161 private func bootstrapBuiltinTools() {
162     let builtins: [ToolDefinition] = [
163         ToolDefinition(
164             id: "hermes.memory.search",
165             displayName: "Memory Search",
166             description: "Search short-term and long-term memory by keyword.",
167             capabilities: [.readMemory],
168             requiredTier: .readonly,
169             agentRoles: [.explore, .plan, .verify],
170             inputSchema: ["query": "string", "limit": "integer"],
171             contextTags: ["memory", "search"]
172         ),
173         ToolDefinition(
174             id: "hermes.memory.observe",
175             displayName: "Observe",
176             description: "Write a new observation to memory.",
177             capabilities: [.writeMemory],
178             requiredTier: .standard,
179             agentRoles: [.execute],
180             inputSchema: ["category": "string", "content": "any", "metadata": "object"],
181             contextTags: ["memory", "write"]
182         ),
183         ToolDefinition(
184             id: "hermes.context.topic",
185             displayName: "Current Topic",
186             description: "Retrieve the detected topic of the current conversation window.",
187             capabilities: [.readContext],
188             requiredTier: .readonly,
189             agentRoles: [.explore, .plan, .verify, .execute],
190             inputSchema: [:],
191             contextTags: ["context"]
192         ),
193         ToolDefinition(
194             id: "hermes.dream.trigger",
195             displayName: "Trigger Dream",
196             description: "Manually trigger a mini-dream consolidation cycle.",
197             capabilities: [.triggerDream, .writeMemory],
198             requiredTier: .privileged,
199             agentRoles: [.execute],
200             inputSchema: [:],
201             contextTags: ["dream", "maintenance"]
202         ),
203         ToolDefinition(
204             id: "hermes.session.read",
205             displayName: "Read Session State",
206             description: "Read crash-safe session state including token usage.",
207             capabilities: [.readSession],
208             requiredTier: .readonly,
209             agentRoles: [.explore, .plan, .verify],
210             inputSchema: [:],
211             contextTags: ["session"]
212         ),
213         ToolDefinition(
214             id: "hermes.session.write",
215             displayName: "Write Session State",
216             description: "Persist session state snapshot to disk.",
217             capabilities: [.writeSession],
218             requiredTier: .privileged,
219             agentRoles: [.execute],
220             inputSchema: ["state": "object"],
221             contextTags: ["session"]
222         ),
223         ToolDefinition(
224             id: "hermes.suggestions.refresh",
225             displayName: "Refresh Suggestions",
226             description: "Force-refresh the Kairos proactive suggestion cache.",
227             capabilities: [.readMemory, .modifyUI],
228             requiredTier: .standard,
229             agentRoles: [.execute],

```

```

230 inputSchema: [:],
231 contextTags: ["suggestions", "ui"]
232 },
233 ]
234 builtins.forEach { register($0) }
235 }
236
237 // MARK: - Security module bootstrap (18 modules)
238
239 private func bootstrapSecurityModules() {
240     securityModules = [
241         // 1. Tool must be enabled
242         SecurityModule(name: "tool_enabled") { tool, _ in tool.isEnabled },
243         // 2. Session must be active (not terminated)
244         SecurityModule(name: "session_active") { _, ctx in ctx.tokenBudgetRemaining >= 0 },
245         // 3. Network tools require foreground
246         SecurityModule(name: "network_foreground") { tool, ctx in
247             !tool.capabilities.contains(.networkAccess) || ctx.isInForeground
248         },
249         // 4. Destructive writes require privileged tier
250         SecurityModule(name: "write_tier") { tool, ctx in
251             !tool.capabilities.contains(.writeMemory) || ctx.sessionTier >= .standard
252         },
253         // 5. File writes require privileged tier
254         SecurityModule(name: "file_write_tier") { tool, ctx in
255             !tool.capabilities.contains(.fileWrite) || ctx.sessionTier >= .privileged
256         },
257         // 6. Subagent execution requires privileged tier
258         SecurityModule(name: "subagent_tier") { tool, ctx in
259             !tool.capabilities.contains(.executeSubagent) || ctx.sessionTier >= .privileged
260         },
261         // 7. Dream trigger requires Kairos to not be in the middle of a cycle
262         SecurityModule(name: "dream_not_running") { tool, ctx in
263             !tool.capabilities.contains(.triggerDream) || !ctx.kairosActive
264         },
265         // 8. Token budget must be > 10% of baseline to allow privileged ops
266         SecurityModule(name: "budget_privileged_floor") { tool, ctx in
267             tool.requiredTier < .privileged || ctx.tokenBudgetRemaining > 1_000
268         },
269         // 9. UI modifications require foreground
270         SecurityModule(name: "ui_foreground") { tool, ctx in
271             !tool.capabilities.contains(.modifyUI) || ctx.isInForeground
272         },
273         // 10. Restricted tools must have explicit metadata flag
274         SecurityModule(name: "restricted_explicit") { tool, ctx in
275             tool.requiredTier < .restricted || (ctx.metadata["explicitRestricted"] as? Bool == true)
276         },
277         // 11. Session writes disallowed in readonly sessions
278         SecurityModule(name: "session_write_tier") { tool, ctx in
279             !tool.capabilities.contains(.writeSession) || ctx.sessionTier >= .privileged
280         },
281         // 12. Context reads always allowed
282         SecurityModule(name: "context_read_allowed") { tool, _ in
283             !tool.capabilities.contains(.readContext) || true
284         },
285         // 13. Memory reads always allowed
286         SecurityModule(name: "memory_read_allowed") { tool, _ in
287             !tool.capabilities.contains(.readMemory) || true
288         },
289         // 14. Tool id must be reverse-domain format
290         SecurityModule(name: "id_format") { tool, _ in
291             tool.id.contains(".")
292         },
293         // 15. Tool must have a non-empty description
294         SecurityModule(name: "description_present") { tool, _ in
295             !tool.description.isEmpty
296         },
297         // 16. Input schema must be present (even if empty) for all write tools
298         SecurityModule(name: "write_schema_present") { tool, _ in
299             !tool.capabilities.contains(.writeMemory) || !tool.inputSchema.isEmpty
300         },
301         // 17. Budget floor for any execution in standard tier
302         SecurityModule(name: "budget_standard_floor") { tool, ctx in
303             tool.requiredTier < .standard || ctx.tokenBudgetRemaining > 100
304         },
305         // 18. Dream trigger restricted to execute role only
306         SecurityModule(name: "dream_execute_role_only") { tool, ctx in

```

